

Programmer's guide: Dessinons un mouton

Brian Lacroix 20220098

March 2026

Table des matières

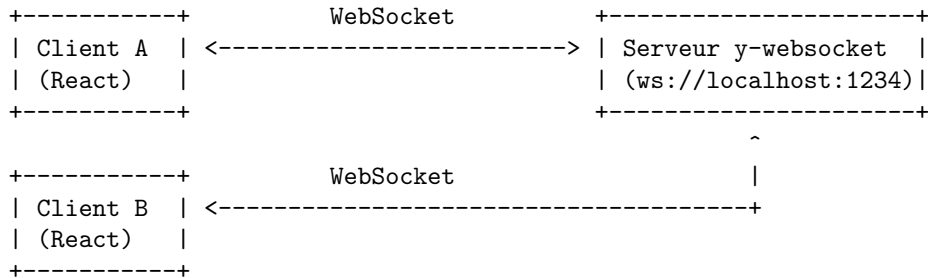
1. Vue d'ensemble du système
2. Stack technologique et motivations des choix
3. Architecture des composants
 4. Modèle de données
 5. Architecture event-driven
 6. Flux de dessin au runtime
7. Collaboration en temps réel
 8. Rendu graphique
 9. Système de tests
10. Glossaire
11. Annexes

1 Vue d'ensemble du système

Dessinons un mouton est une application web de dessin collaboratif. Elle permet à plusieurs utilisateurs de dessiner simultanément sur un canevas partagé, en voyant en temps réel les traits des autres participants.

Le système comprend deux composants principaux :

- Le client React : interface utilisateur, logique de dessin, synchronisation.
- Le serveur YJS WebSocket (**y-websocket**) : relais de la synchronisation entre clients. Il ne contient aucune logique métier et se contente de propager les changements d'un client à l'autre.



1.1 Par où commencer pour poursuivre le développement ?

Cette section fournit les informations essentielles pour toute personne souhaitant reprendre ou étendre le projet.

1.1.1 Sources et documentation

La documentation officielle des principales technologies utilisées dans ce projet est disponible aux adresses suivantes :

- React : <https://react.dev/learn>
- TypeScript : <https://www.typescriptlang.org/docs/>
- Konva : <https://konvajs.org/docs/index.html>
- React-Konva : <https://konvajs.org/docs/react/index.html>
- YJS : <https://docs.yjs.dev>
- Mediabunny : <https://mediabunny.dev/guide/introduction>
- Jest : <https://jestjs.io/docs/getting-started>

1.1.2 Développeur

Ce projet a été développé par Brian Lacroix (brian.lacroix@student.unamur.be), étudiant en troisième année de bachelier en sciences informatiques à l'Université de Namur.

1.1.3 Cahier des charges

Le cahier des charges a été fourni par Monsieur Pierre Poitier. Il est disponible en annexe D de ce document.

1.1.4 Commanditaire

Le projet a été réalisé à la demande de Monsieur Pierre Poitier.

2 Stack technologique et motivations des choix

Technologie	Version	Rôle	Pourquoi ce choix
React	18.3.1	UI et état local	Standard de l'industrie ; les hooks permettent une séparation nette de la logique métier et du rendu
TypeScript	4.9.5	Typage statique	Détection d'erreurs à la compilation, indispensable pour un modèle de données complexe (6 types de strokes, 7 événements)
Konva / react-konva	10.2.0 / 18.2.14	Rendu canvas 2D	API déclarative sur canvas HTML5 qui supporte <code>globalCompositeOperation</code> pour l'effet gomme
YJS	13.6.29	Synchronisation CRDT	Résolution automatique des conflits sans serveur central intelligent ; fonctionne hors ligne et réconcilie les états à la reconnexion
y-websocket	2.0.4	Transport réseau pour YJS	Serveur WebSocket léger fourni directement par l'écosystème YJS, prêt à l'emploi sans configuration
mediabunny	1.39.0	Assemblage vidéo WebM	Successeur officiel de webm-muxer (même auteur) ; permet d'encoder des frames via l'API WebCodecs du navigateur avec des timestamps arbitraires, sans attendre en temps réel
Create React App	5.0.1	Build & test	Configuration Jest intégrée sans configuration complexe
Jest + Testing Library	jest-dom 6.9.1 / react 13.4.0	Tests	Environnement de test standard pour React ; <code>renderHook</code> permet de tester les hooks en isolation totale

3 architecture des composants

3.1 Arborescence des fichiers avec motivations

```
src/
+-- types/
|   +-- drawing.ts           - Layer, Stroke (6 variantes), Position, BrushSettings
|   +-- events.ts           - DrawingEvent : type union des 7 actions possibles
|
+-- constants/
|   +-- tools.ts            - TOOL_TYPES, BRUSH_CONFIG, CANVAS_CONFIG
|
+-- utils/
|   +-- events.ts           - applyEvent() et invertEvent() : cœur de la logique undo
|   +-- geometry.ts         - createInitialStroke() et updateStrokeWithPosition()
|
+-- hooks/
|   +-- useCollaboration.ts - Hub central : YJS, dispatch, undo, previews distants
|   +-- useDrawing.ts       - Gestion des événements pointeur, preview local
|   +-- useLayers.ts        - Operations utilisateur sur les layers
|   +-- useRecorder.ts      - Capture de frames et encodage vidéo WebM via WebCodecs
|
+-- components/
|   +-- canvas/
|   |   +-- CanvasStage.tsx  - Assemblage Konva : layers confirmées + previews
|   |   +-- StrokeRenderer.tsx - Rendu d'un stroke (React.memo)
|   +-- controls/
|   |   +-- Toolbar.tsx      - Undo, export PNG, enregistrement vidéo
|   |   +-- BrushControls.tsx - Couleur, taille, opacité
|   |   +-- ShapeControls.tsx - Sélection des outils formes
|   +-- layers/
|   |   +-- LayersPanel.tsx   - Panneau droit listant les layers
|   |   +-- LayerItem.tsx     - Ligne individuelle d'une layer
|   +-- ui/
|   |   +-- Button.tsx / ColorPicker.tsx / Slider.tsx / ToolButton.tsx
|
+-- testUtils/
|   +-- fixtures.ts          - Données de test partagées entre toutes les suites
|
+-- __tests__/
|   +-- utils/events.test.ts
|   +-- utils/geometry.test.ts
|   +-- hooks/useLayers.test.ts
|   +-- hooks/useCollaboration.test.ts
|   +-- hooks/useDrawing.test.ts
|
+-- App.tsx                 - Composant racine : assemble les hooks et les composants
+-- index.tsx               - Point d'entrée CRA
+-- setupTests.ts           - Polyfill crypto.randomUUID() pour Jest
+-- docker-compose.yml
+-- Dockerfile.client
+-- Dockerfile.server
+-- .dockerignore
```

3.2 Séparation des responsabilités

L'architecture distingue trois niveaux de responsabilité :

Les utils contiennent des fonctions pures sans effet de bord. Elles ne dépendent d'aucun état React et sont entièrement testables de façon unitaire. C'est le moteur logique de l'application.

Les hooks contiennent l'état et les effets de bord (connexion YJS, écoute des événements). Ils dépendent des utils mais pas des composants. Ce choix permet de tester la logique métier sans rendre de composant.

Les composants sont responsables uniquement de l'affichage. Ils reçoivent leurs données et callbacks via des props et ne contiennent aucune logique métier.

4 Modèle de données

4.1 Layer

Une Layer est un calque contenant une liste ordonnée de strokes. Les layers sont rendues dans l'ordre du tableau, la première est en bas, la dernière est au-dessus.

```
interface Layer {  
  id: string;          // Identifiant unique (ex: "layer-1", "layer-1748392")  
  name: string;        // Nom affiché dans le panneau  
  visible: boolean;    // Contrôle la visibilité dans le rendu Konva  
  strokes: Stroke[];  
}
```

4.2 Stroke - type union

Un Stroke est l'union discriminée de 6 interfaces, toutes héritant d'un BaseStroke :

```
interface BaseStroke {  
  id: string;          // crypto.randomUUID() - unique globalement  
  color: string;       // Format CSS (ex: "#ff0000", "rgba(0,0,0,1)")  
  size: number;        // Épaisseur du trait en pixels, entre 1 et 60  
  opacity: number;     // Opacité du trait, entre 0.05 et 1  
}
```

Type	Champs spécifiques	Rendu konva
Brush	points: number[] (paires x,y)	Line avec lineCap = "round"
eraser	points: number[]	Line avec globalCompositeOperation = "destination-out"
rectangle / square	x, y, width, height, startX, startY	Rect
circle	x, y, radius, startX, startY	Circle
ellipse	x, y, width, height, startX, startY	Ellipse (radiusX/Y = width/height ÷ 2)
line	points: number[], startX, startY	Line

Motivation du type union : TypeScript peut vérifier à la compilation que tous les cas sont traités dans les switch. Un oubli de cas dans applyEvent ou StrokeRenderer devient une erreur de compilation plutôt qu'un bug silencieux au runtime.

4.3 DrawingEvent - les 7 événements atomiques

```
type DrawingEvent =  
  | { type: "ADD_STROKE";      layerId: string; stroke: Stroke }  
  | { type: "REMOVE_STROKE";   layerId: string; stroke: Stroke }  
  | { type: "ADD_LAYER";       layer: Layer }  
  | { type: "REMOVE_LAYER";    layer: Layer }  
  | { type: "RENAME_LAYER";    layerId: string; oldName: string; newName: string }  
  | { type: "MOVE_LAYER";      layerId: string; fromIndex: number; toIndex: number }  
  | { type: "TOGGLE_VISIBILITY"; layerId: string };
```

Chaque événement est auto-descriptif : il contient toutes les données nécessaires pour être appliqué ou inversé, sans avoir besoin de consulter l'état courant. Ceci est essentiel pour `invertEvent()` et pour la sérialisation YJS.

5 Architecture event-driven

5.1 Principe

Toute modification de l'état passe par un événement `DrawingEvent`. Aucune mutation directe du tableau de `layers` n'est autorisée depuis les composants. Ce choix garantit que :

- L'état est toujours reconstituable à partir d'une liste d'événements.
- Chaque action est réversible via `invertEvent`.
- La synchronisation YJS peut se faire en appliquant les effets de manière déterministe.

5.2 `applyEvent(layers, event) → layers`

Fonction pure qui prend l'état courant et un événement, et retourne le nouvel état sans jamais modifier le tableau en entrée.

```
layers -----> layers'  
              applyEvent(layers, ADD_STROKE)
```

5.3 `invertEvent(event) → event`

Retourne l'événement symétrique, permettant à undo de défaire n'importe quelle action :

Événement	Inverse
ADD STROKE	REMOVE STROKE (mêmes données)
REMOVE STROKE	ADD STROKE (mêmes données)
ADD LAYER	REMOVE LAYER
REMOVE LAYER	ADD LAYER
RENAME LAYER	RENAME LAYER avec <code>oldName</code> et <code>newName</code> inversés
MOVE LAYER	MOVE LAYER avec <code>fromIndex</code> et <code>toIndex</code> inversés
TOGGLE VISIBILITY	TOGGLE VISIBILITY (son propre inverse)

5.4 Historique et Undo

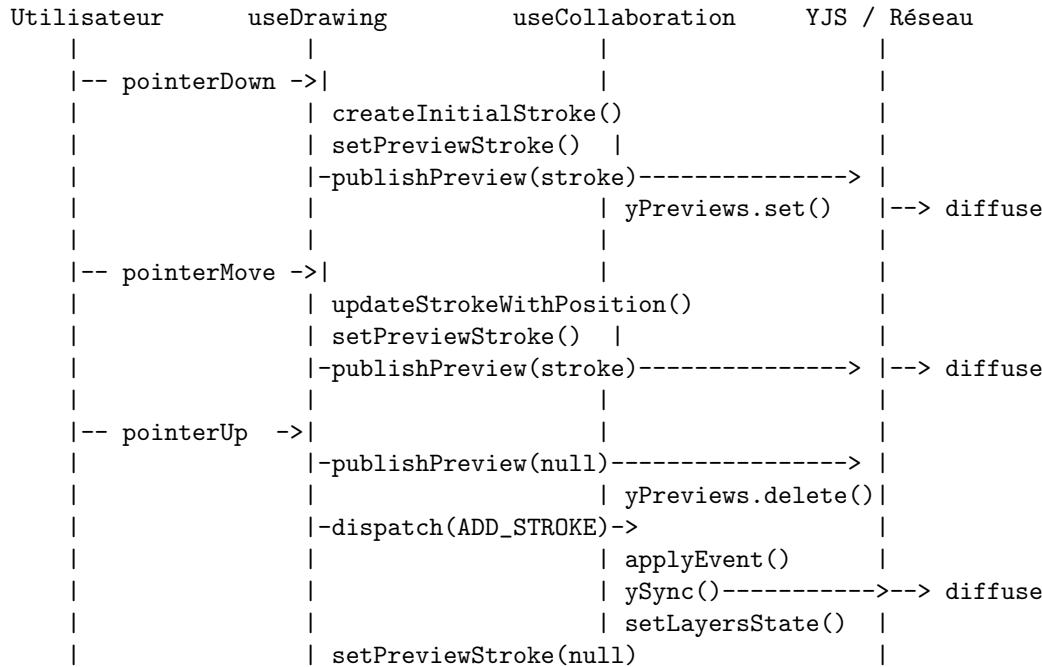
L'historique est une pile LIFO de `DrawingEvent[]` limitée à 50 entrées (MAX HISTORY), stockée dans un `useRef` dans `useCollaboration` pour éviter des re-renders inutiles.

```
dispatch(ADD_STROKE)    historique: [ADD_STROKE]  
dispatch(ADD_LAYER)    historique: [ADD_STROKE, ADD_LAYER]  
undo()                  applique invertEvent(ADD_LAYER) = REMOVE_LAYER  
                        historique: [ADD_STROKE]
```

Propriété fondamentale : les previews (traits en cours de dessin) ne transitent jamais par l'historique. `dispatch` n'est appelé qu'au `pointerUp`. Un `undo` retire donc toujours un trait complet, jamais un point intermédiaire.

6 Flux de dessin au runtime

6.1 Séquence d'un trait complet (brush)



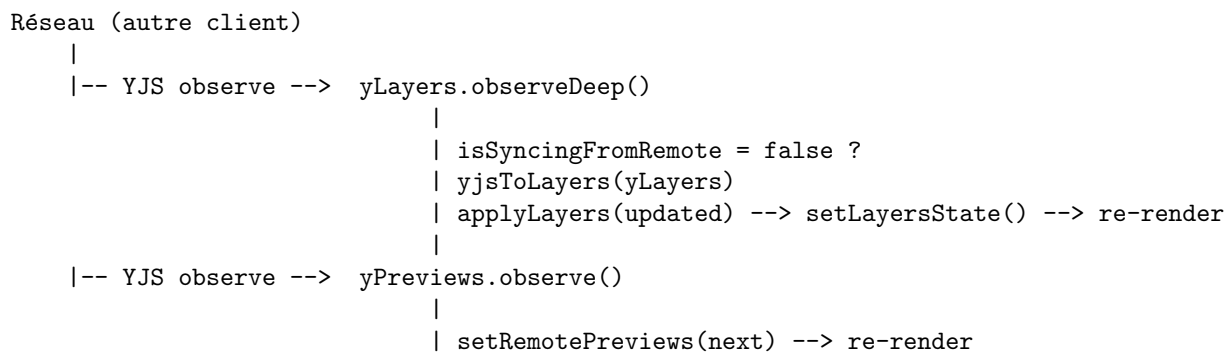
6.2 Rôle du `layersRef`

`useCollaboration` maintient en parallèle un `layersRef` (React ref) et un `layers` (state). Ce double stockage est nécessaire car :

- Le state React (`layers`) déclenche les re-renders du composant.
- La ref (`layersRef`) donne accès à la valeur courante sans fermeture stale dans les callbacks `dispatch` et `undo`, qui sont mémorisés avec `useCallback`.

Sans cette ref, `dispatch` lirait toujours la valeur de `layers` capturée au moment de sa création, pas la valeur actuelle.

6.3 Réception d'un événement distant



Le flag `isSyncingFromRemote` évite une boucle infinie : quand le client local écrit dans YJS (`ySync`), l'observer YJS se déclenche. Sans ce flag, cela appellerait `applyLayers` une deuxième fois inutilement.

7 Collaboration en temps réel

7.1 YJS et les CRDT

YJS utilise des CRDT (Conflict-free Replicated Data Types) : des structures de données conçues pour être modifiées simultanément par plusieurs clients sans nécessiter de coordination centrale. Les conflits sont résolus automatiquement de façon déterministe.

L'application utilise deux structures YJS :

- `Y.ArrayY.Map` nommée "layers" : contient les layers persistées (toutes les données confirmées).
- `Y.Mapstring` nommée "previews" : contient les traits en cours de dessin de chaque client (clé = clientID YJS, valeur = JSON du stroke).

7.2 Sérialisation des strokes

Les strokes d'une layer sont sérialisés en JSON et stockés comme une seule valeur dans le `Y.Map` de la layer :

```
yLayer.set("strokes", JSON.stringify(layer.strokes))
```

Motivation : YJS gère bien la convergence au niveau des structures (qui ajoute/retire quel `Y.Map`), mais les strokes changent très fréquemment pendant le dessin. Les traiter comme une valeur atomique JSON simplifie la logique de synchronisation et évite des conflits complexes sur des tableaux YJS imbriqués.

7.3 Preview en temps réel

Les previews (traits en cours, pas encore confirmés) transitent via `yPreviews` et sont intentionnellement séparées de l'état persistant. Chaque client écrit sa propre preview sous son `clientID`. Les autres clients lisent toutes les previews sauf la leur.

À la fin d'un trait (`pointerUp`), la preview est supprimée de `yPreviews` et le stroke confirmé est écrit dans `yLayers` via `dispatch`.

8 Rendu graphique

8.1 CanvasStage

`CanvasStage` assemble le rendu `Konva`. Pour chaque layer, il affiche dans cet ordre :

1. Les strokes confirmés de la layer (`layer.strokes`).
2. La preview locale du client courant (si `previewStroke` existe et correspond à la layer active).
3. Les previews distantes des autres clients pour cette layer (filtrées depuis `remotePreviews`).

8.2 StrokeRenderer

`StrokeRenderer` est mémorisé avec `React.memo` pour éviter les re-renders inutiles. Il délègue à `Konva` selon le type de stroke :

- brush / eraser → `Line` ; l'eraser utilise `globalCompositeOperation="destination-out"` qui efface les pixels au lieu de les peindre.
- rectangle / square → `Rect`
- circle → `Circle`
- ellipse → `Ellipse` avec `radiusX = width/2` et `radiusY = height/2`
- line → `Line`

Tous les éléments graphiques ont `listening=false` car l'application n'utilise pas les événements `Konva` sur les strokes - les événements pointeur sont capturés sur le `Stage` parent.

8.3 Enregistrement vidéo (useRecorder)

useRecorder permet de capturer une vidéo timelapse du dessin au format WebM, incluant les actions de tous les participants. Il expose deux fonctions : startRecording et stopAndDownload.

Capture : dès que l'enregistrement est actif, un `setInterval` prend un snapshot du canvas toutes les 50 ms via `stage.toDataURL()`, indépendamment de la source du changement visuel (local ou distant, preview ou stroke confirmé). Tous les participants sont donc traités de façon identique.

Avant l'encodage, les frames consécutives identiques sont supprimées. Cela élimine automatiquement les pauses entre les traits : la vidéo ne contient que les moments où quelqu'un dessine.

Encodage : au stopAndDownload, les frames sont encodées sans attente en temps réel via l'API WebCodecs du navigateur et assemblées dans un fichier .webm par mediabunny. Un CanvasSource dessine chaque frame sur un canvas offscreen, et les timestamps sont calculés pour que la vidéo dure exactement targetDurationS secondes :

```
const frameDurationS = targetDurationS / frames.length;
await videoSource.add(i * frameDurationS, frameDurationS);
```

9 Système de tests

9.1 Organisation et couverture

```
src/__tests__/
+--- utils/events.test.ts          - applyEvent (7 cas) + invertEvent
+--- utils/geometry.test.ts        - createInitialStroke + updateStroke
+--- hooks/useLayers.test.ts       - addLayer, deleteLayer, moveLayer, rename, toggle
+--- hooks/useCollaboration.test.ts - dispatch, undo, skipHistory, publishPreview
+--- hooks/useDrawing.test.ts      - flux pointerDown -> pointerMove -> pointerUp
```

Total : 120 tests, 5 suites, 0 échec.

9.2 Stratégie de mock pour YJS

YJS et y-websocket sont des modules ESM que Jest (CRA) ne peut pas transpiler nativement. Le hook useCollaboration est entièrement mocké dans ses tests par une implémentation en mémoire qui reproduit fidèlement la logique métier : dispatch, undo LIFO, skipHistory, publishPreview.

Avantage : les tests sont indépendants du réseau et du serveur YJS, ils s'exécutent en isolation totale et de façon déterministe.

9.3 Simulation crypto

Jest tourne dans Node.js où crypto.randomUUID() n'est pas global par défaut. setupTests.ts expose le webcrypto de Node comme global avant chaque suite :

```
const { webcrypto } = require('crypto');
Object.defineProperty(global, 'crypto', { value: webcrypto, writable: true });
```

10 Glossaire

Terme	Définition
CRDT	Conflict-free Replicated Data Type : structure de données pouvant être modifiée simultanément par plusieurs nœuds sans conflit
Event-driven	Architecture où les changements d'état sont décrits par des événements immuables plutôt que par des mutations directes
Stroke	Élément graphique dessiné sur le canevas (trait, forme, gomme)
Layer	Calque contenant une liste ordonnée de strokes
Preview	Trait en cours de dessin, non encore confirmé dans l'historique
Dispatch	Fonction qui applique un événement à l'état et le synchronise via YJS
Undo	Annulation de la dernière action via <code>invertEvent</code> sur le dernier événement de l'historique
layersRef	Référence React (<code>useRef</code>) maintenant la valeur courante de layers pour les callbacks mémorisés
isSyncingFromRemote	Flag booléen évitant les boucles infinies dans l'observer YJS lors d'une écriture locale
LIFO	Last In, First Out - structure de pile utilisée pour l'historique undo
WebCodecs	API navigateur permettant d'encoder des frames vidéo avec des timestamps arbitraires, sans contrainte temps réel

11 Annexes

11.1 Annexes A: Démarrer le projet

11.1.1 avec le terminal

assurez-vous d'être dans "project/dessc2" avant d'exécuter ces lignes de commande

```
# 1. Installer les dépendances:
```

```
npm install
```

```
# 2. Démarrer le serveur YJS (dans un terminal dédié):
```

```
node server.js
```

```
# 3. Démarrer l'application React (dans un second terminal):
```

```
npm start
```

si vous voulez exécuter les tests, faites simplement:

```
npm test
```

11.1.2 avec docker

assurez-vous d'être dans "project/dessc2" avant d'exécuter cette ligne de commande

```
#1. build le dockerfile:
```

```
docker-compose up --build
```

```
#2. démarrer le container dans docker quand vous souhaitez l'utiliser
```

Pour tester la collaboration : ouvrir <http://localhost:3000> dans deux onglets ou deux navigateurs différents.

11.2 Annexe B: Ajouter un nouvel outil de dessin

1. Ajouter la constante dans `constants/tools.ts` (TOOL-TYPES et TOOL-CONFIGS).
2. Ajouter l'interface du stroke dans `types/drawing.ts` et l'inclure dans le type union `Stroke`.

3. Gérer la création initiale dans `utils/geometry.ts` → `createInitialStroke()`.
4. Gérer la mise à jour pendant le déplacement dans `utils/geometry.ts` → `updateStrokeWithPosition()`.
5. Ajouter le cas de rendu dans `components/canvas/StrokeRenderer.tsx`.
6. Ajouter les tests dans `__tests__/utils/geometry.test.ts` et `__tests__/hooks/useDrawing.test.ts`.

11.3 Annexe C: Ajouter un nouvel événement `DrawingEvent`

1. Ajouter le type dans `types/events.ts`.
2. Ajouter le cas dans `utils/events.ts` → `applyEvent()`.
3. Ajouter le cas inverse dans `utils/events.ts` → `invertEvent()`.
4. Ajouter la fonction déclenchante dans `hooks/useLayers.ts` si c'est une action utilisateur.
5. Ajouter les tests dans `__tests__/utils/events.test.ts`.

11.4 Annexe D: le cahier des charges

Dessignons un mouton

Site de dessin collaboratif

Acronyme – DESSC

Client – Pierre Poitier <pierre.poitier@unamur.be>

Description

Dessiner seul c'est bien. Dessiner à plusieurs c'est mieux ! L'objectif de ce projet est de réaliser **un site web où l'on peut dessiner à plusieurs**, en même temps ! Il s'agirait d'un site assez sobre avec : des profils utilisateurs, un espace de dessin et un tchat. Un utilisateur devra être capable de dessiner, d'effacer ses dessins et également de mettre des commentaires dans le dessin. La page web, c'est-à-dire la feuille de dessin, devra également être navigable (déplacement, zoom, etc.). Les utilisateurs devront pouvoir voir le dessin des autres en temps réel ainsi que de pouvoir créer des calques et leur donner des noms. Un défi est que toutes les modifications d'un dessin devront être enregistrées, de sorte qu'à la fin on puisse lancer un « replay » de son évolution. Il est donc encouragé d'utiliser une approche event-driven.

Le point central de ce projet et la qualité de son implémentation, aspect sur lequel une attention particulière sera demandée. Il s'agit d'une opportunité de mettre en pratique des design patterns (observables, proxies, event sourcing, etc) ainsi que mettre en œuvres les bonnes pratiques que vous avez apprises (documentation, testing, etc.). L'idée est simple, mais une attention particulière sera demandée pour l'implémentation et les requirements non-fonctionnels.

Technologies employées

Java, Python, JavaScript, HTML, CSS

Le choix des technologies doit être discuté avec le client en amont.

Requirements fonctionnels

Degré d'importance : Très important [***] - Important [**] - Secondaire [*]

- [***] Être capable de dessiner sur une page web
- [***] Synchronisation en direct des dessins de tout le monde
- [***] Possibilité d'annuler sa dernière action (CTRL - Z)
- [***] Système de calques
- [**] Tchat écrit
- [**] Formes basiques (lignes, rectangles, ovales, courbe)
- [**] Pinceaux (taille, pattern, opacité, couleur)
- [*] Rejouer l'historique des dessins sur la page
- [*] Inscription et connexion
- [*] Sauvegarde du dessin dans différents formats
- [*] Pouvoir incorporer des images dans le dessin

Requirements non-fonctionnels

Degré d'importance : Très important [***] - Important [**] - Secondaire [*]

- [***] Documentation technique
- [***] Modularité et qualité du code
- [**] Aspect général
- [**] Fluidité
- [**] Simplicité
- [*] Animations